

UNIDAD 8

Redes neuronales

Fundamentos para comprender LSTM y GNN aplicadas a blockchain

Qué aprende una red neuronal

Función parametrizada

Una red transforma variables de entrada en una salida mediante operaciones cuyos pesos aprende a partir de ejemplos.

Aprendizaje supervisado

Cada ejemplo combina características X con un objetivo y . El entrenamiento ajusta parámetros para reducir una pérdida.

Generalización

El valor del modelo se evalúa sobre observaciones que quedaron fuera del entrenamiento.

Entradas, capas ocultas y salida

Componente	Función	Ejemplo en blockchain
Entrada	Recibe características medidas	Retorno, volumen, gas o atributos de un nodo
Capas ocultas	Construyen representaciones intermedias	Combinaciones aprendidas de actividad y mercado
Salida	Produce una estimación del objetivo	Retorno, probabilidad o volatilidad
Parámetros	Pesos y sesgos aprendibles	Valores que se ajustan durante el entrenamiento

La neurona artificial

$$z = w_1x_1 + w_2x_2 + \dots + b$$

$$a = \varphi(z)$$

Pesos w

Modulan la contribución de cada entrada. El signo y la escala dependen de la representación del dato.

Sesgo b y activación φ

El sesgo desplaza la respuesta. La activación introduce una transformación no lineal.

Ejemplo numérico de una neurona

$$\mathbf{x} = (0,5; -1,0) \quad \mathbf{w} = (0,8; -0,2) \quad \mathbf{b} = 0,1$$

$$z = 0,8 \times 0,5 + (-0,2) \times (-1,0) + 0,1 = 0,7$$

$$\text{ReLU}(z) = 0,7$$

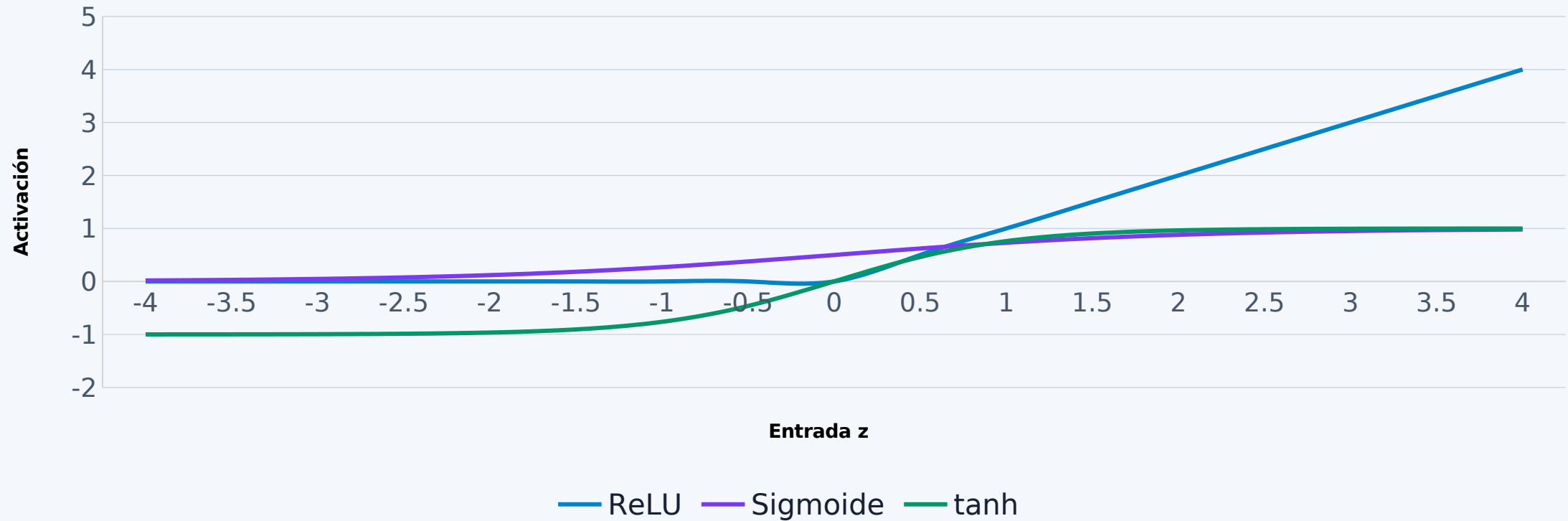
Lectura del cálculo

Las entradas ya están escaladas. La salida corresponde a una activación interna.

Alcance

Ejemplo matemático ilustrativo. Estos números no provienen de datos de BTC o ETH.

Funciones de activación



ReLU: $\max(0, z)$. Sigmoid: valores entre 0 y 1. tanh: valores entre -1 y 1.

Propagación hacia adelante

$$\mathbf{h} = \text{ReLU}(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$$

$$\hat{\mathbf{y}} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2$$

Cálculo de la predicción

La entrada atraviesa las capas con los parámetros actuales. Para regresión, la última salida puede ser lineal.

Profundidad

Agregar capas aumenta la capacidad del modelo y también la necesidad de datos y regularización.

Objetivo, salida y pérdida

Problema	Salida habitual	Pérdida o evaluación
Retorno futuro	Valor real	MSE o MAE
Dirección	Logit o probabilidad	Entropía cruzada binaria
Volatilidad	Estimación positiva	Pérdida sobre varianza o log-varianza
Clase de un nodo	Logits por clase	Entropía cruzada con máscara de etiquetas

Ejemplo de error cuadrático

$$y = (1,0; -1,0) \quad \hat{y} = (0,5; 0,5)$$

$$\text{MSE} = [(1,0 - 0,5)^2 + (-1,0 - 0,5)^2] / 2$$

$$\text{MSE} = (0,25 + 2,25) / 2 = 1,25$$

Unidades

Si los retornos se expresan en puntos porcentuales, el MSE se expresa en puntos porcentuales al cuadrado.

Interpretación

El segundo error pesa más porque la pérdida eleva al cuadrado su magnitud.

Descenso del gradiente

$$\theta \text{ nuevo} = \theta \text{ actual} - \eta \nabla \theta L$$

$$w \text{ actual} = 0,8 \quad \eta = 0,1 \quad \partial L / \partial w = 0,3$$

$$w \text{ nuevo} = 0,8 - 0,1 \times 0,3 = 0,77$$

Tasa de aprendizaje η

Controla la magnitud del paso. Un valor alto puede desestabilizar el entrenamiento.

Gradiente

Indica cómo cambia la pérdida al variar un parámetro en el punto actual.

Retropropagación y entrenamiento

Predicción y pérdida

La red calcula $\hat{y}$ y compara esa salida con el objetivo observado.

Regla de la cadena

La retropropagación calcula derivadas a través de las operaciones de cada capa.

Actualización

El optimizador utiliza los gradientes para modificar pesos y sesgos. El ciclo se repite con nuevos lotes.

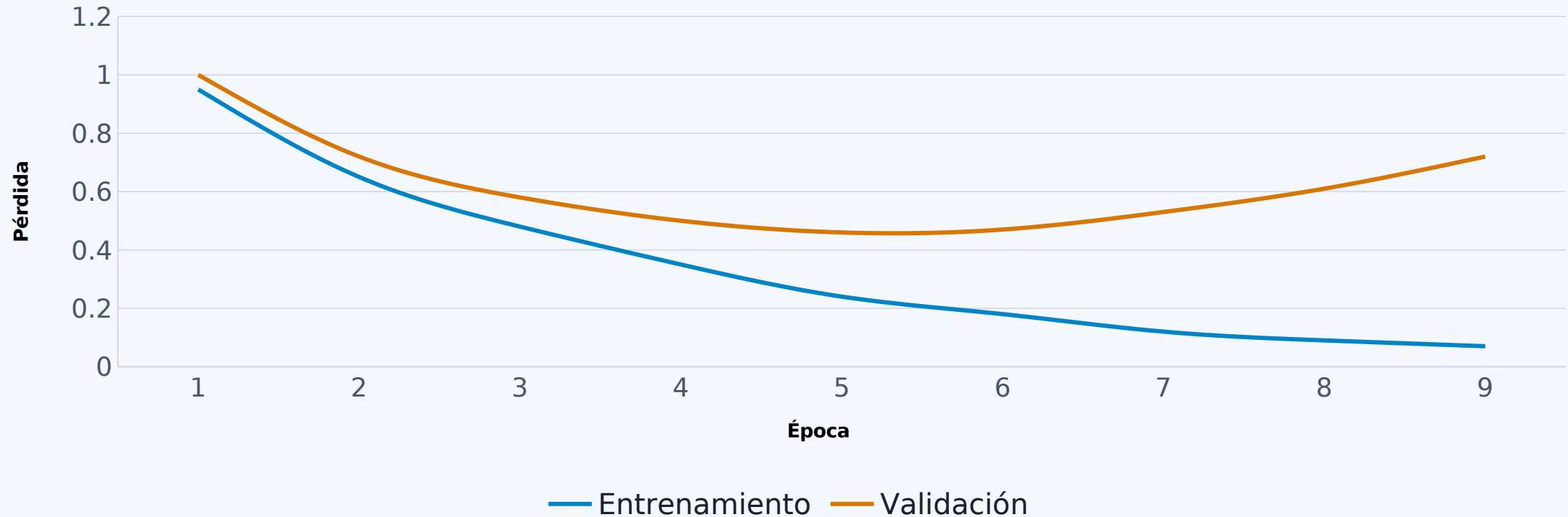
Hiperparámetros del entrenamiento

Concepto	Definición	Decisión práctica
Epoch	Recorrido de los ejemplos de entrenamiento	Seleccionar con validación y early stopping
Batch	Lote utilizado para una actualización	Equilibrar ruido, memoria y costo
Learning rate	Escala del paso del optimizador	Ajustar en el período de validación
Hidden size	Dimensión de la representación oculta	Comparar capacidad y generalización

Entrenamiento, validación y prueba

Conjunto	Uso	Restricción temporal
Entrenamiento	Ajustar pesos y transformaciones	Sólo información disponible antes del corte
Validación	Elegir arquitectura e hiperparámetros	Período posterior al entrenamiento
Prueba final	Estimar generalización	Período reservado hasta cerrar el diseño

Sobreajuste y early stopping



Curvas sintéticas. La pérdida de validación mínima aparece en la época 5.

Regularización y controles

Capacidad del modelo

Comparar una red pequeña antes de aumentar capas o unidades. Registrar cantidad de parámetros.

Regularización

Dropout y penalizaciones sobre pesos reducen algunas formas de sobreajuste. Su intensidad se elige en validación.

Estabilidad

Usar varias semillas, inspeccionar curvas y conservar el mejor checkpoint de validación.

Familias de redes

Arquitectura	Estructura utilizada	Aplicación del curso
MLP	Vector de variables o ventana aplanada	Baseline tabular de mercado y on-chain
RNN, LSTM y GRU	Orden de una secuencia	Retorno, volatilidad y actividad futura
GNN	Vecindarios y relaciones de un grafo	Clasificación y representaciones de transacciones
Modelo híbrido	Grafo por período y secuencia	Evaluar información temporal y estructural

Actividad de comprensión

Problema

Una red recibe retorno, volumen y gas de los últimos 30 días para estimar el retorno del día siguiente.

Análisis

Identificar entradas, objetivo, salida, pérdida y baseline. Explicar cómo separar entrenamiento y prueba.

Discusión

¿Qué evidencia permitiría justificar una LSTM frente a una MLP con la misma ventana?

Bibliografía y documentación

Goodfellow, Bengio y Courville (2016). Deep Learning, capítulo 8

<https://www.deeplearningbook.org/contents/optimization.html>

Hochreiter y Schmidhuber (1997). Long Short-Term Memory

<https://www.bioinf.jku.at/publications/older/2604.pdf>

PyTorch. LSTM: ecuaciones, estados y dimensiones

<https://docs.pytorch.org/docs/stable/generated/torch.nn.LSTM.html>

Cho et al. (2014). Learning Phrase Representations using RNN Encoder-Decoder

<https://arxiv.org/abs/1406.1078>

Kipf y Welling (2017). Semi-Supervised Classification with GCN

<https://arxiv.org/abs/1609.02907>